

Assignment 14

TCP, UDP and IPV4 Header Analysis using Wireshark

Name: Arnav Singhal

MIS: 612203012

TY B. Tech. Comp. Div 1

Batch: T3

1. Introduction

This report presents a practical analysis of three core networking protocol headers — TCP (Transmission Control Protocol), UDP (User Datagram Protocol), and IPv4 (Internet Protocol version 4) — captured and examined using Wireshark, a widely used open-source network packet analyser.

Understanding the structure of protocol headers is fundamental to networking: each field encodes specific control information that determines how data is routed, delivered and verified across a network. By capturing live traffic and inspecting the header fields directly, this exercise bridges theory and real-world network behaviour.

2. TCP Header Analysis

2.1 Overview of TCP

TCP is a connection-oriented transport layer protocol defined in RFC 793. It provides reliable, ordered and error-checked delivery of data between applications. Before any data is exchanged, TCP establishes a connection using a three-way handshake (SYN → SYN-ACK → ACK), and it closes the connection using a four-way termination sequence.

2.2 Wireshark Capture - TCP Packet List

No.	Time	Source	Destination	Protocol	Length	Info
56	2.8011554...	2401:4900:8fcb::...	2404:6800:4009::...	TCP	1454	39720 → 443 [ACK] Seq=1 Ack=1 Win=10167 Len=1368 TSval=920782845 TSecr=2320954634 [1
57	2.8011678...	2401:4900:8fcb::...	2404:6800:4009::...	TCP	1454	39720 → 443 [ACK] Seq=1369 Ack=1 Win=10167 Len=1368 TSval=920782845 TSecr=2320954634
58	2.8011693...	2401:4900:8fcb::...	2404:6800:4009::...	TLSV1...	444	Application Data
59	2.8012448...	2401:4900:8fcb::...	2404:6800:4009::...	TCP	1454	39720 → 443 [ACK] Seq=3095 Ack=1 Win=10167 Len=1368 TSval=920782845 TSecr=2320954634
60	2.8012519...	2401:4900:8fcb::...	2404:6800:4009::...	TCP	1454	39720 → 443 [ACK] Seq=4463 Ack=1 Win=10167 Len=1368 TSval=920782845 TSecr=2320954634
61	2.8012530...	2401:4900:8fcb::...	2404:6800:4009::...	TLSV1...	303	Application Data
62	2.8116614...	2404:6800:4009::...	2401:4900:8fcb::...	TCP	86	443 → 39720 [ACK] Seq=1 Ack=1369 Win=2560 Len=0 TSval=2320958320 TSecr=920782845
65	2.8116614...	2404:6800:4009::...	2401:4900:8fcb::...	TCP	86	443 → 39720 [ACK] Seq=1 Ack=3095 Win=2556 Len=0 TSval=2320958320 TSecr=920782845
66	2.8116615...	2404:6800:4009::...	2401:4900:8fcb::...	TCP	86	443 → 39720 [ACK] Seq=1 Ack=4463 Win=2560 Len=0 TSval=2320958320 TSecr=920782845
67	2.8116615...	2404:6800:4009::...	2401:4900:8fcb::...	TCP	86	443 → 39720 [ACK] Seq=1 Ack=5831 Win=2556 Len=0 TSval=2320958320 TSecr=920782845
68	2.8116615...	2404:6800:4009::...	2401:4900:8fcb::...	TCP	86	443 → 39720 [ACK] Seq=1 Ack=6048 Win=2556 Len=0 TSval=2320958320 TSecr=920782845
103	2.8909615...	2404:6800:4009::...	2401:4900:8fcb::...	TLSV1...	630	Application Data
104	2.8909619...	2404:6800:4009::...	2401:4900:8fcb::...	TLSV1...	454	Application Data
105	2.8912164...	2401:4900:8fcb::...	2404:6800:4009::...	TCP	86	39720 → 443 [ACK] Seq=6048 Ack=913 Win=10167 Len=0 TSval=920782935 TSecr=2320958402
106	2.8940659...	2404:6800:4009::...	2401:4900:8fcb::...	TLSV1...	117	Application Data
107	2.8940661...	2404:6800:4009::...	2401:4900:8fcb::...	TLSV1...	125	Application Data
108	2.8951369...	2401:4900:8fcb::...	2404:6800:4009::...	TCP	86	39720 → 443 [ACK] Seq=6048 Ack=983 Win=10167 Len=0 TSval=920782939 TSecr=2320958403
109	2.8956980...	2401:4900:8fcb::...	2404:6800:4009::...	TLSV1...	125	Application Data
110	2.9021877...	2404:6800:4009::...	2401:4900:8fcb::...	TCP	86	443 → 39720 [ACK] Seq=983 Ack=6087 Win=2560 Len=0 TSval=2320958413 TSecr=920782940
155	3.7911632...	2404:6800:4009::...	2401:4900:8fcb::...	TLSV1...	171	Application Data
156	3.7911637...	2404:6800:4009::...	2401:4900:8fcb::...	TLSV1...	117	Application Data
157	3.7911637...	2404:6800:4009::...	2401:4900:8fcb::...	TLSV1...	125	Application Data
158	3.7915156...	2401:4900:8fcb::...	2404:6800:4009::...	TCP	86	51168 → 443 [ACK] Seq=1 Ack=156 Win=424 Len=0 TSval=2616667398 TSecr=1204468006
159	3.7925014...	2401:4900:8fcb::...	2404:6800:4009::...	TLSV1...	121	Application Data
160	3.7925352...	2401:4900:8fcb::...	2404:6800:4009::...	TLSV1...	125	Application Data
165	3.8016033...	2404:6800:4009::...	2401:4900:8fcb::...	TCP	86	443 → 51168 [ACK] Seq=156 Ack=36 Win=945 Len=0 TSval=1204468075 TSecr=2616667399
166	3.8016034...	2404:6800:4009::...	2401:4900:8fcb::...	TCP	86	443 → 51168 [ACK] Seq=156 Ack=75 Win=945 Len=0 TSval=1204468076 TSecr=2616667399
171	3.8255404...	2401:4900:8fcb::...	2404:6800:4009::...	TCP	94	48476 → 443 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4204366885 TSecr=0
172	3.8340699...	2404:6800:4009::...	2401:4900:8fcb::...	TCP	94	443 → 48476 [ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1380 SACK_PERM TSval=28272696

Frame 65: 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface
Ethernet II, Src: ServercomPri_3d:68:80 (b4:a7:c6:3d:68:80), Dst: AzureWav
Internet Protocol Version 6, Src: 2404:6800:4009:815::200e, Dst: 2401:4900
Transmission Control Protocol, Src Port: 443, Dst Port: 39720, Seq: 1, Ack

2.3 TCP Three-Way Handshake

A key feature of TCP is the three-way handshake used to establish a reliable connection before data transfer. The following screenshot shows the SYN, SYN-ACK, and ACK packets as captured in Wireshark.

No.	Time	Source	Destination	Protocol	Length	Info
158	3.791515...	2401:4900:8fcb::	2404:6800:4009::	TCP	86	51168 → 443 [ACK] Seq=1 Ack=156 Win=424 Len=0 TSval=2616667398 TSecr=1204468075
159	3.792501...	2401:4900:8fcb::	2404:6800:4009::	TLSv...	121	Application Data
160	3.792535...	2401:4900:8fcb::	2404:6800:4009::	TLSv...	125	Application Data
165	3.801603...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 51168 [ACK] Seq=156 Ack=36 Win=945 Len=0 TSval=1204468075 TSecr=2616667398
166	3.801603...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 51168 [ACK] Seq=156 Ack=75 Win=945 Len=0 TSval=1204468076 TSecr=2616667398
171	3.825549...	2401:4900:8fcb::	2404:6800:4009::	TCP	94	48476 → 443 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4204366885 TSecr=2616667398
172	3.834069...	2404:6800:4009::	2401:4900:8fcb::	TCP	94	443 → 48476 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1380 SACK_PERM TSval=4204366893 TSecr=282726917
173	3.834154...	2401:4900:8fcb::	2404:6800:4009::	TCP	86	48476 → 443 [ACK] Seq=1 Ack=1 Win=64896 Len=0 TSval=4204366893 TSecr=282726917
174	3.835162...	2401:4900:8fcb::	2404:6800:4009::	TCP	14	48476 → 443 [ACK] Seq=1 Ack=1 Win=64896 Len=1368 TSval=4204366894 TSecr=282726917
175	3.835181...	2401:4900:8fcb::	2404:6800:4009::	TLSv...	962	Client Hello (SNI=signaler-pa.clients6.google.com)
176	3.845223...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 48476 [ACK] Seq=1 Ack=1369 Win=268544 Len=0 TSval=282726917 TSecr=4204366894
177	3.845223...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 48476 [ACK] Seq=1 Ack=2245 Win=267776 Len=0 TSval=282726918 TSecr=4204366894
178	3.848452...	2404:6800:4009::	2401:4900:8fcb::	TLSv...	15	Server Hello, Change Cipher Spec, Application Data
179	3.848505...	2401:4900:8fcb::	2404:6800:4009::	TCP	86	48476 → 443 [ACK] Seq=2245 Ack=1497 Win=63488 Len=0 TSval=4204366908 TSecr=282726917
180	3.849141...	2401:4900:8fcb::	2404:6800:4009::	TLSv...	176	Change Cipher Spec, Application Data
181	3.849633...	2401:4900:8fcb::	2404:6800:4009::	TLSv...	738	Application Data, Application Data
182	3.857998...	2404:6800:4009::	2401:4900:8fcb::	TLSv...	11	Application Data, Application Data
183	3.858282...	2401:4900:8fcb::	2404:6800:4009::	TLSv...	117	Application Data
184	3.861248...	2404:6800:4009::	2401:4900:8fcb::	TLSv...	117	Application Data
185	3.871680...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 48476 [ACK] Seq=2552 Ack=3018 Win=267264 Len=0 TSval=282726945 TSecr=4204366908

2.4 Expanded TCP Header Detail

No.	Time	Source	Destination	Protocol	Length	Info
64	2.811661...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 39720 [ACK] Seq=1 Ack=1369 Win=2560 Len=0 TSval=2320958320 TSecr=920782
65	2.811661...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 39720 [ACK] Seq=1 Ack=3095 Win=2556 Len=0 TSval=2320958320 TSecr=920782
66	2.811661...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 39720 [ACK] Seq=1 Ack=4463 Win=2560 Len=0 TSval=2320958320 TSecr=920782
67	2.811661...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 39720 [ACK] Seq=1 Ack=5831 Win=2556 Len=0 TSval=2320958320 TSecr=920782
68	2.811661...	2404:6800:4009::	2401:4900:8fcb::	TCP	86	443 → 39720 [ACK] Seq=1 Ack=6048 Win=2556 Len=0 TSval=2320958320 TSecr=920782

Frame 66: 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface wlo1, id 0

Ethernet II, Src: ServercomPri_3d:68:80 (b4:a7:c6:3d:68:80), Dst: AzureWaveTec_af:ad:31 (a8:41:f4:29:58:00)

Internet Protocol Version 6, Src: 2404:6800:4009:815::200e, Dst: 2401:4900:8fcb:1052:5192:aca2:18c::

Transmission Control Protocol, Src Port: 443, Dst Port: 39720, Seq: 1, Ack: 4463, Len: 0

Source Port: 443

Destination Port: 39720

[Stream index: 0]

[Conversation completeness: Incomplete (12)]

[TCP Segment Len: 0]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 482438169

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 4463 (relative ack number)

Acknowledgment number (raw): 357303550

1000 = Header Length: 32 bytes (8)

Flags: 0x010 (ACK)

Window: 2560

[Calculated window size: 2560]

[Window size scaling factor: -1 (unknown)]

Checksum: 0xe191 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

[Timestamps]

[SEQ/ACK analysis]

2.5 TCP Header Field Analysis

The following table documents each field observed in the captured TCP header:

Field	Size (bits)	Observed Value	Description
Source Port	16	443	Port used by the sending application
Destination Port	16	39720	Identifies the receiving application/service
Sequence Number	32	1	Tracks byte order; used to reassemble out-of-order segments
Acknowledgment No.	32	4463	Next byte expected from sender; confirms receipt of prior data
Header Length	4	32 bytes	Length of TCP header in 32-bit words (min 20 bytes, max 60 bytes)
Reserved	3	000	Unused bits; must be zero as per RFC 793
Flags (Control)	9	0x010 (ACK)	SYN: initiate connection; ACK: acknowledge; FIN: close; RST: reset; PSH: push data
Window Size	16	2560	Number of bytes sender is willing to receive (flow control)
Checksum	16	0xe191	Error detection; covers header + data + pseudo-header

Urgent Pointer	16	0	Only active when URG flag is set; points to urgent data end
Options	0–320	NOP, NOP, Timestamps	Optional fields — common: MSS, Window Scale, SACK, Timestamps

2.6 Observations

- The Window Size value indicates the receiver's buffer capacity for flow control.
- Relative sequence numbers (used by Wireshark by default) simplify readability; actual values are large 32-bit integers.
- The ACK flag is set in nearly all packets after the initial SYN, confirming reliable delivery.
- TCP Options such as NOP and Timestamps were visible in the SYN packet.

3. UDP Header Analysis

3.1 Overview of UDP

UDP is a connectionless transport layer protocol defined in RFC 768. Unlike TCP, UDP does not establish a connection before sending data, does not guarantee delivery or ordering, and does not provide error recovery. This makes UDP significantly faster and more suitable for real-time applications such as DNS lookups, video streaming, VoIP, and online gaming, where low latency is preferred over guaranteed delivery.

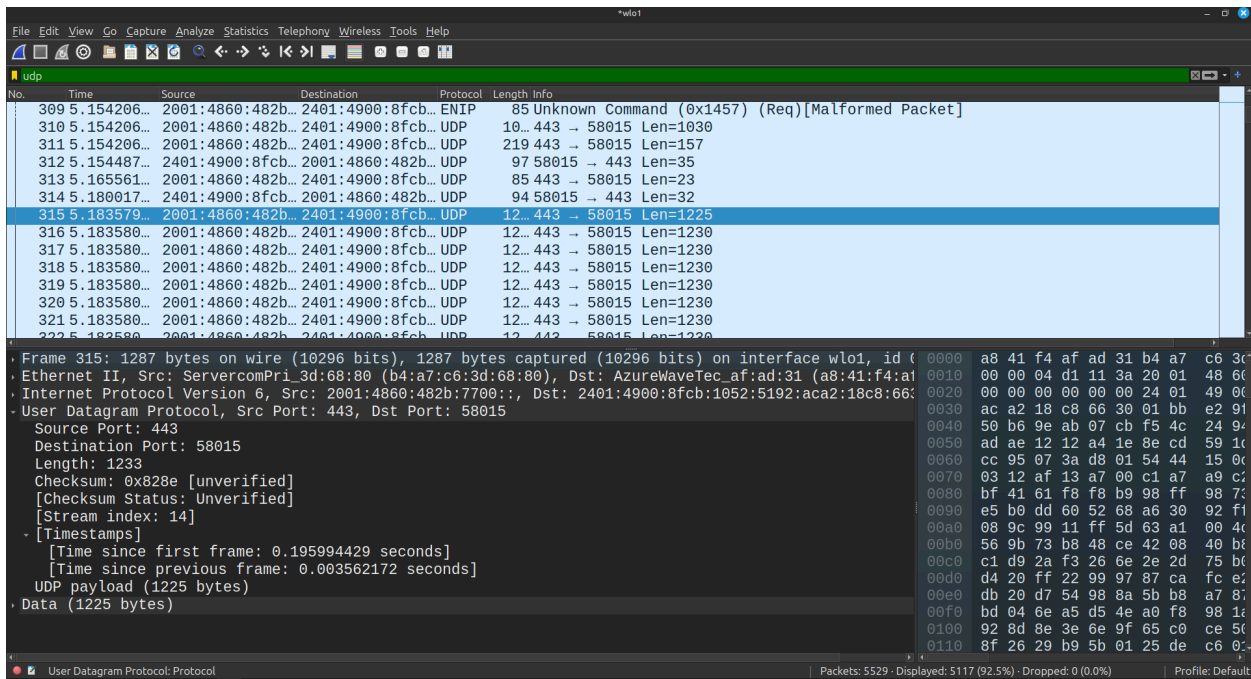
3.2 Wireshark Capture - UDP Packet List

The image shows a Wireshark capture of UDP traffic on interface wlo1. The packet list pane displays 34 packets, all of which are UDP. The packets are numbered 320 to 341. The source and destination IP addresses are 2001:4860:482b:: and 2401:4900:8fcb:: respectively. The source port is 443 and the destination port is 58015. The length of the packets is 1230 bytes. The packet details pane shows the structure of a UDP packet, including the Ethernet II header, Internet Protocol Version 6 header, User Datagram Protocol header, and Data (33 bytes). The packet bytes pane shows the raw data in hexadecimal and ASCII format.

No.	Time	Source	Destination	Protocol	Length	Info
320	5.183580...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
321	5.183580...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
322	5.183580...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
323	5.183679...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
324	5.183679...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
325	5.183679...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
326	5.183679...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
327	5.183679...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
328	5.183679...	2001:4860:482b::	2401:4900:8fcb::	UDP	776	443 → 58015 Len=714
329	5.183916...	2401:4900:8fcb::	2001:4860:482b::	UDP	93	58015 → 443 Len=31
330	5.183971...	2401:4900:8fcb::	2001:4860:482b::	UDP	93	58015 → 443 Len=31
331	5.184002...	2401:4900:8fcb::	2001:4860:482b::	UDP	93	58015 → 443 Len=31
332	5.184028...	2401:4900:8fcb::	2001:4860:482b::	UDP	93	58015 → 443 Len=31
333	5.184055...	2401:4900:8fcb::	2001:4860:482b::	UDP	93	58015 → 443 Len=31
334	5.184095...	2401:4900:8fcb::	2001:4860:482b::	UDP	93	58015 → 443 Len=31
335	5.184129...	2401:4900:8fcb::	2001:4860:482b::	UDP	93	58015 → 443 Len=31
336	5.211100...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1223
337	5.211100...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
338	5.211100...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
339	5.211100...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
340	5.211100...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230
341	5.211100...	2001:4860:482b::	2401:4900:8fcb::	UDP	12...	443 → 58015 Len=1230

Frame 193: 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface wlo1, id 0
Ethernet II, Src: ServercomPri_3d:68:80 (b4:a7:c6:3d:68:80), Dst: AzureWaveTec_af:ad:31 (a8:41:f4:a1:ad:31)
Internet Protocol Version 6, Src: 2404:6800:4009:818::200e, Dst: 2401:4900:8fcb:1052:5192:aca2:18c8:1052
User Datagram Protocol, Src Port: 443, Dst Port: 54521
Data (33 bytes)
0000 a8 41 f4 af ad 31 b4 a7 c6 3d
0010 00 00 00 29 11 3b 24 04 c8 00
0020 00 00 00 00 20 0e 24 01 49 00
0030 ac a2 18 c8 66 30 01 bb d4 f9
0040 63 be 9a ec 42 99 3f 29 e6 88
0050 ad 18 ac 84 d9 05 06 81 f0 a9

3.3 Expanded UDP Header Detail



3.4 UDP Header Field Analysis

UDP has a minimal 8-byte header consisting of only four fields:

Field	Size (bits)	Observed Value	Description
Source Port	16	443	Sending application's port
Destination Port	16	58015	Identifies the receiving application/service
Length	16	1223	Total length of UDP header + data payload in bytes (min 8)
Checksum	16	0x828e	Optional in IPv4; covers header + data + pseudo-header for error detection

3.5 TCP vs UDP Comparison

Feature	TCP	UDP
Connection	Connection-oriented (handshake required)	Connectionless (no handshake)
Reliability	Guaranteed delivery with retransmission	No guarantee; packets may be lost
Ordering	Packets delivered in order	No ordering; may arrive out of order
Header Size	20-60 bytes	8 bytes (fixed)
Speed	Slower due to overhead	Faster. minimal overhead
Error Handling	Full error checking + recovery	Checksum only (optional in IPv4)
Use Cases	HTTP, HTTPS, FTP, email (SMTP)	DNS, VoIP, video streaming, gaming

3.6 Observations

- The UDP header is only 8 bytes, significantly smaller than TCP's minimum 20 bytes.
- No sequence or acknowledgment numbers are present, confirming UDP's stateless nature.
- The short length field in the captured packet reflects a typical small DNS query payload.

4. IPv4 Header Analysis

4.1 Overview of IPv4

IPv4 (Internet Protocol version 4) operates at the Network Layer (Layer 3) of the OSI model and is responsible for addressing, routing and fragmentation of packets across networks. Every TCP and UDP packet is encapsulated inside an IPv4 packet. The IPv4 header is defined in RFC 791 and has a minimum size of 20 bytes.

4.2 Wireshark Capture - IPv4 Packet List

The image shows a Wireshark network traffic capture. The top pane displays a list of captured packets, and the bottom pane shows a detailed view of a selected packet (Frame 222).

Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
90	0.487985...	192.168.1.13	8.8.8.8	ICMP	98	Echo (ping) request id=0x6289, seq=1/256, ttl=64 (reply in 10)
100	0.496468...	8.8.8.8	192.168.1.13	ICMP	98	Echo (ping) reply id=0x6289, seq=1/256, ttl=118 (request in 9)
231	1.266580...	192.168.1.1	192.168.1.255	UDP	363	43327 → 9995 Len=321
271	1.489548...	192.168.1.13	8.8.8.8	ICMP	98	Echo (ping) request id=0x6289, seq=2/512, ttl=64 (reply in 28)
281	1.498004...	8.8.8.8	192.168.1.13	ICMP	98	Echo (ping) reply id=0x6289, seq=2/512, ttl=118 (request in 27)
161	3.798153...	192.168.1.13	192.168.1.1	DNS	102	Standard query 0x72e4 AAAA signaler-pa.clients6.google.com OPT
162	3.798337...	192.168.1.13	192.168.1.1	DNS	102	Standard query 0x1219 A signaler-pa.clients6.google.com OPT
163	3.798502...	192.168.1.13	192.168.1.1	DNS	102	Standard query 0xbc6e HTTPS signaler-pa.clients6.google.com OPT
168	3.818444...	192.168.1.1	192.168.1.13	DNS	130	Standard query response 0x72e4 AAAA signaler-pa.clients6.google.com AAAA 2404
169	3.821552...	192.168.1.1	192.168.1.13	DNS	118	Standard query response 0x1219 A signaler-pa.clients6.google.com A 142.250.20
170	3.824778...	192.168.1.1	192.168.1.13	DNS	152	Standard query response 0xbc6e HTTPS signaler-pa.clients6.google.com SOA ns1
217	4.495575...	192.168.1.13	192.168.1.1	DNS	107	Standard query 0x82d9 AAAA drivefrontend-pa.clients6.google.com OPT
218	4.495673...	192.168.1.13	192.168.1.1	DNS	107	Standard query 0xcdd A drivefrontend-pa.clients6.google.com OPT
219	4.495749...	192.168.1.13	192.168.1.1	DNS	107	Standard query 0x173e HTTPS drivefrontend-pa.clients6.google.com OPT
220	4.503675...	192.168.1.1	192.168.1.13	DNS	135	Standard query response 0x82d9 AAAA drivefrontend-pa.clients6.google.com AAAA
221	4.506978...	192.168.1.1	192.168.1.13	DNS	123	Standard query response 0xcdd A drivefrontend-pa.clients6.google.com A 142.2
222	4.506978...	192.168.1.1	192.168.1.13	DNS	157	Standard query response 0x173e HTTPS drivefrontend-pa.clients6.google.com SOA
366	5.233660...	192.168.1.13	192.168.1.1	DNS	85	Standard query 0x55d7 AAAA lh3.google.com OPT
367	5.233747...	192.168.1.13	192.168.1.1	DNS	85	Standard query 0x704b A lh3.google.com OPT
368	5.233809...	192.168.1.13	192.168.1.1	DNS	85	Standard query 0xb00e HTTPS lh3.google.com OPT
369	5.241677...	192.168.1.1	192.168.1.13	DNS	133	Standard query response 0x55d7 AAAA lh3.google.com CNAME lh2.l.google.com AA
372	5.244960...	192.168.1.1	192.168.1.13	DNS	121	Standard query response 0x704b A lh3.google.com CNAME lh2.l.google.com A 142
373	5.244960...	192.168.1.1	192.168.1.13	DNS	155	Standard query response 0xb00e HTTPS lh3.google.com CNAME lh2.l.google.com S
374	5.245228...	192.168.1.13	192.168.1.1	DNS	87	Standard query 0x65f5 HTTPS lh2.l.google.com OPT
381	5.254767...	192.168.1.1	192.168.1.13	DNS	127	Standard query response 0x65f5 HTTPS lh2.l.google.com SOA ns1.google.com OPT

Frame 222: 157 bytes on wire (1256 bits), 157 bytes captured (1256 bits) on interface wlo1, id 0

Ethernet II, Src: ServercomPri_3d:68:80 (b4:a7:c6:3d:68:80), Dst: AzureWaveTec_af:ad:31 (a8:41:f4:a1:ad:31)

Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.13

User Datagram Protocol, Src Port: 53, Dst Port: 48090

Domain Name System (response)

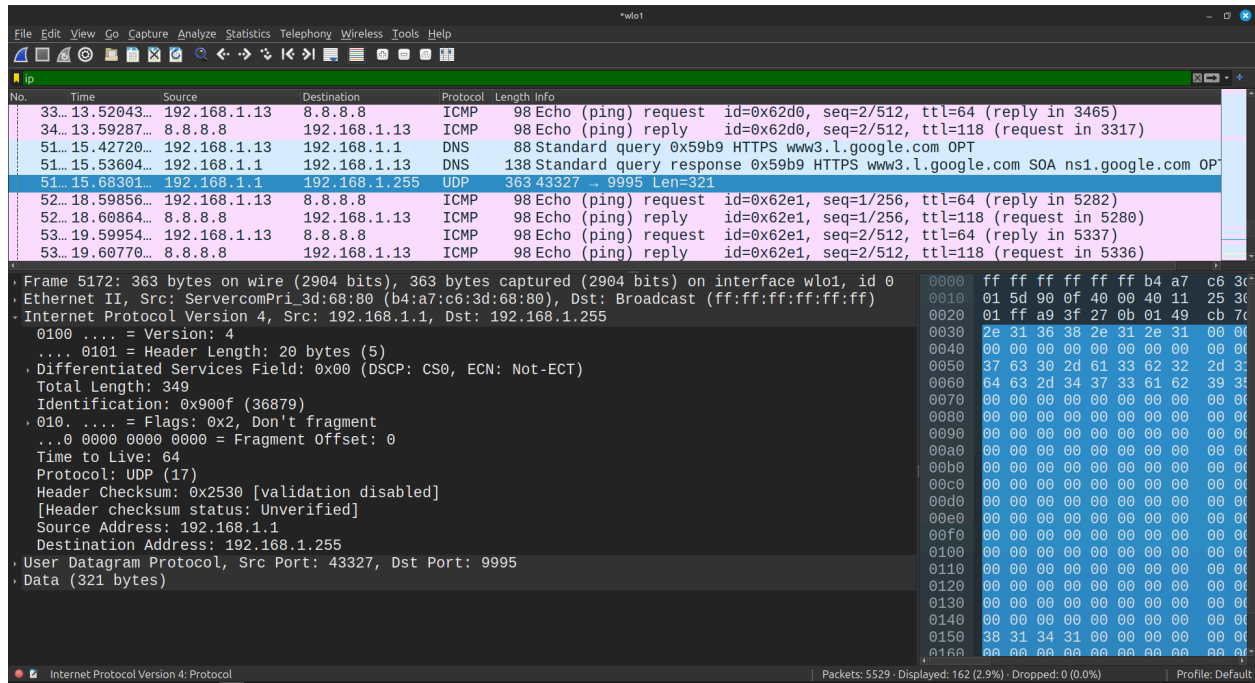
Packet Details:

- Frame 222: 157 bytes on wire (1256 bits), 157 bytes captured (1256 bits) on interface wlo1, id 0
- Ethernet II, Src: ServercomPri_3d:68:80 (b4:a7:c6:3d:68:80), Dst: AzureWaveTec_af:ad:31 (a8:41:f4:a1:ad:31)
- Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.13
- User Datagram Protocol, Src Port: 53, Dst Port: 48090
- Domain Name System (response)

Packet Bytes:

```
0000  a8 41 f4 af ad 31 b4 a7 c6 3d 68 80 00 00 00 00
0010  00 8f a7 d1 40 00 40 11 0f 2e 00 00 00 00 00 00
0020  01 0d 00 35 bb da 00 7b ca 31 00 00 00 00 00 00
0030  00 00 00 01 00 01 10 64 72 6c 00 00 00 00 00 00
0040  74 65 6e 64 2d 70 61 08 63 6c 00 00 00 00 00 00
0050  06 67 6f 6f 67 6c 65 03 63 61 00 00 00 00 00 00
0060  c0 28 00 06 00 01 00 00 00 00 00 00 00 00 00 00
```

4.3 Expanded IPv4 Header Detail



4.4 IPv4 Header Field Analysis

The following table documents every field in the IPv4 header as observed in the capture:

Field	Size (bits)	Observed Value	Description
Version	4	4	IP version number - always 4 for IPv4
Header Length (IHL)	4	5 (20 bytes)	Header length in 32-bit words; 5 = 20 bytes (no options)
DSCP	6	0x00 (CS0)	Differentiated Services Code Point - Quality of Service (QoS) marking

ECN	2	0x0 (Non-ECT)	Explicit Congestion Notification - signals network congestion
Total Length	16	349	Total size of IP packet including header + data, in bytes
Identification	16	0x900f (36879)	Unique ID for packet; used to group fragments belonging together
Flags	3	0x2 (DF=1)	DF: Don't Fragment; MF: More Fragments; bit 0 reserved
Fragment Offset	13	0	Position of this fragment in the original datagram (0 if not fragmented)
TTL	8	64	Time To Live - decremented at each hop; 0 = packet dropped. 64=Linux/Mac, 128=Windows
Protocol	8	17	Encapsulated protocol: 6=TCP, 17=UDP, 1=ICMP
Header Checksum	16	0x2530	Error detection for the IPv4 header only (not the payload)

Source IP	32	192.168.1.1	Sender's IPv4 address
Destination IP	32	192.168.255	Receiver's IPv4 address
Options	0–320	None	Optional; rarely used - examples: record route, timestamp

4.5 Observations

- The TTL value reveals the likely operating system of the sender: 128 suggests Windows, 64 suggests Linux or macOS.
- The DF (Don't Fragment) flag was set to 1, meaning the packet must not be fragmented by intermediate routers.
- The Protocol field value of 17 confirms UDP encapsulation; a value of 6 would indicate UDP.
- The Identification field is unique per packet, essential for reassembling fragmented datagrams.
- The Header Checksum protects only the IPv4 header. The payload (TCP/UDP) has its own checksum.
- DSCP and ECN were both 0, indicating standard best-effort traffic with no congestion signalling.

5. Summary

The following table summarises the key characteristics across all three protocol headers analysed in this report:

Attribute	TCP	UDP	IPv4
OSI Layer	Transport (L4)	Transport (L4)	Network (L3)
Header Size	20–60 bytes	8 bytes	20–60 bytes
Connection	Connection-oriented	Connectionless	Connectionless
Reliability	Guaranteed	Best-effort	Best-effort
Key Fields	Seq, Ack, Flags, Window	Src Port, Dst Port, Len	TTL, Protocol, IPs, Flags
RFC Reference	RFC 793	RFC 768	RFC 791

6. Conclusion

This experiment provided hands-on exposure to the internal structure of TCP, UDP, and IPv4 protocol headers through live packet capture using Wireshark. Several important observations emerged from the analysis:

- TCP's complexity (sequence numbers, flags, window size) directly enables its reliability and flow control features.
- UDP's minimal 8-byte header explains its speed advantage, making it the protocol of choice for latency-sensitive applications like DNS and streaming.
- IPv4 provides the foundation for both, encapsulating transport layer segments and handling routing via addressing and TTL.
- The TTL field in IPv4 is a practical indicator of the source operating system and also prevents infinite routing loops.
- Wireshark's protocol dissector makes it possible to observe each field in real-time, bridging the gap between protocol specification and practical network behaviour.